Project Record - part 3 of 8

Utensil Theories Transfer Protocol, 17 to 21 October 2026. This part holds: 4. The owner's ideas, in their own words, and what was built from each.

4. The owner's ideas, in their own words, and what was built from each

Each idea below gives the owner's words, the plain meaning, what the AI built and what the tests found. Quotes are copied exactly as typed, typos included. "Message N" is the Nth message in the numbered list of the owner's messages.

All tests used tiny models trained from scratch: about 3.4 million parameters, able to see the last 64 words. A result here does not change how a large AI understands anyone.

4.1 Starting point: comprehension for the AI, not reading people

The owner first pasted a script that scored "human error" in text: "Evaluate and enhance if possible". The AI fixed its bugs (`resonance/resonance_engine.py`, 16 passing tests). Then came message 5: "I am trying to teach actual "comprehension" and "understanding" for A.I. not about people".

That set the objective. The script reads people, so it was dropped as the framing. Its "wavelength" is a text fingerprint (a code computed from the text), not a measurement.

4.2 Rules plus human error as the source of meaning

- **Words (message 6, 17 Oct):** "The rules are the secret of understanding. The things we do wrong naturally and have to be told not to do them is something ai must mimic to grasp reality as a human does." Also: "If a person has never seen (born blind) do the truly understand what color is?"
- **Meaning:** understanding forms where rules meet the mistakes that break them. A reader without senses can still learn meaning from how words relate.
- **Built:** `experiments/bible_understanding.py` (experiments 1-4), a word-meaning model of the King James Bible (31,102 verses). Then `consequences_test.py` and `tiny_transformer.py` (experiments 5-6), at the owner's "yes try the consequences test next" and "yes try the tiny transformer next".
- **Results:**
  - Color without sight worked: 10 of 14 matches (71%), against 17% by guessing.
  - Rules alone were not the secret: they scored 0.625 on right and wrong, against 0.709 for the same number of random verses. Removing every rule changed almost nothing (0.948 vs 0.950).
  - The law sharpens wrongness: "covet" weakened from -0.077 to -0.043 without the rules. But 0 of 15 wrong acts flipped to good, so the stories teach it too.
  - Predicting consequences in new stories: chance (45-57%).
  - **What did show up** was the "have to be told" part. Marking right and wrong in the training text lifted judging of new stories to about 69 right per 100, against about 50 unmarked (experiments 8 and 10). This is strong and repeated, but only a side measure: it was never yet pre-registered (fixed in writing before the run) as the main one.

4.3 The shifting word (the owner's central idea)

- **Words (message 9, 17 Oct, 23:00 UTC):**
  - "I will create a new word and give it a variant in its token structure. The idea is based on a syllables rotation sequence rather than a logical one"
  - "[w1=17 -<Next word following it equal to (-13)> = +4 Remaining] next emergence of the word [W1=17+4 -<next 2 words following equaling (00)>= 0 Remaining]"
  - "it must reset to [w1=0] when a clear right or wrong choice exists. If wrong the sequence will begin with a value of (-17) and right a value of (+17)"
  - The variations "to", "too", "two" and "2"

    - The aim: "shift the "coin toss" towards the desired outcome"
- **Meaning:** a made-up word carries a running balance from one use to the next. The words after it add to the balance or use it up, and what is left (the "Remaining") carries forward. A clear right or wrong resets it to +17 or -17. Its spelling changes with the balance, so a reader sees the state in the text. "Shift the coin toss" means beating a 50% guess.
- **Built:** `experiments/shifting_word.py`.
    - The word *velanok* has five spellings, rotations of sha-ru-vel-an-ok, running from `anoksha` (heavy wrong) through `oksharu`, `velanok` (unresolved) and `sharuvel` to `ruvelan` (heavy right).
    - `RESET = 17` on a clear choice: a Bible verdict phrase, or a verse among the 3% most strongly worded at either end. That makes 2,029 reset verses.
    - Word values come from the Bible model's good/evil direction (steal -5, love +6).
- **What the AI changed, and why:**
    - **The recharge ("W1=17+4") was switched off.** With it on, the balance pinned at its cap and almost every verse got an extreme spelling. It was left as the switch `RECHARGE`, untested until the numbers test (4.5).
    - **A fade was added,** `SPEND = 0.5` per valued word, so the balance drifts back toward "unresolved".
    - **Limits were added:** a cap at +/- 51 and a reset at each book start.
    - **Only spellings reached the model, never the number.** Its reader sees letters and punctuation only (see 4.10).
- **Results:**

Experiment	Test	Result
7	20 story pairs, consequences	68% vs 52% scrambled. Later shown to be luck
8	100 new pairs, consequences (pre-registered)	Not confirmed: 51/100
8	100 new pairs, judging (side measure)	67, 72, 67 vs scrambled 56, 42, 60 and unmarked 49, 60, 40
10	Vs a plain good/bad label per verse	Tie: 68.7% vs 67.7%. Not confirmed
11	Long stories, everyday act out of view	Supported once: 59, 56, 63 vs label 54.5, 51, 54 (p = 0.004). A re-run after two fixes; about 65 of 100 pairs tied
11	Long stories, verdict stated	Not confirmed: the label won
12	Exact seed size, 623 unseen verses	Not confirmed: 3.45 bits per verse larger

- **Known defect:** it "carries perfectly but updates poorly". Started on "unresolved", it copied "unresolved" all the way down.
- **Untested:** whether the shared syllables matter (this needs a sub-word reader), and the recharge rule.

4.4 Stories transfer further than rules (Aesop)

- **Words (message 11, 18 Oct):** "Lessons taught through story have more transferable data and relation vectors to give further understanding of things. Its the theory of relatively within wording and comprehension".
- **Built:**
    - `world_stories.py`: 12 public-domain collections, about 727,000 words, using only material published for sharing.
    - `aesop_test.py` version 3, revised twice before any training. It uses 67 Aesop fables never seen in training (1,131 pairs) and trains 16 models.
- **Results:** every pre-registered test was not confirmed. With 50% as guessing, folk tales scored 52.3%, novels 54.1%, shuffled folk tales 53.1% and the Bible 43.6%.
    - **A hint only:** on pairs with no shared-word clue, story models scored about 60% against 54% for novels. Shuffled stories matched them, so the hint points to story vocabulary and setting, not structure.
    - **Not disproved:** the models were too small to test the idea fairly.

4.5 The owner's numbers: 17, 32, 16, 34, 47

- **Words:**

    - Message 9: "17,32,16,34,47 are numbers my subconscious mind has focused on for years , they may just be random noise or maybe useful i dont know".
    - Message 38 (20 Oct): "Add them and test the nunbers "17,32,16,34"=?0=-0=+0=00=["47,"="17+4"]".
    - The owner's clarification (message 39): "I just know my sub conscious mind has had those numbers in that sequence and the stands for me the +4 was a way to direct you to a way to try testing the numbers where I feel like they belong".
- **Meaning:** the numbers matter in that order, and they go where 17 sits in "W1 = 17 + 4 Remaining". That place is the fresh charge the word gets at each appearance, with the Remaining carried on top.
- **Before this:** only 17 was used, as the reset size, and in that design only ratios mattered.
- **What the AI did:**
    - Three independent readers and a judge (AI agents) worked out the design (`NUMBERS_TEST_2026-10-21/design/interpretations_and_design.md`). They found the old `RECHARGE` switch could not test the numbers at all: 16, 32, 34 and 47 all produce identical training text.
    - **Two choices were the AI's own:**
        - each valued word uses up 14.1073% of the charge (`DRAIN`), so that ordinary text settles at exactly 17 + 4 = 21, leaving 4;
        - the word is spelled from the Remaining, as the owner wrote.
    - The zero symbols (?0, -0, +0, 00) are not tested, because the owner has no fixed meaning for them.
- **Built:** `experiments/numbers_test.py`, six versions trained 3 times each (18 tiny models):

Version	Charge at each appearance
`num_charge17`	17 every time (the owner's plain recharge rule)
`num_seq`	17, 32, 16, 34, 47, repeating (the owner's order)
`num_seqrev`	17, 47, 34, 16, 32 (order control)
`num_const29`	29.2, the average (size control)
`num_charge47`	47 every time (written in the "17+4" place)
`num_drain0`	0 (stops the AI's drain from taking credit)

- **The pass/fail rules were fixed before training.** For example, "the owner's numbers matter" means `num_seq` beats `num_charge17` on judging 100 story pairs.
- **Status:**
    - **The dry run passed** on 21 Oct, 02:06-02:08 UTC. All six versions passed the manipulation check (a pre-set check that each rule really changes the text): at most 1.23% of verses at the cap. The stored control scores were reproduced exactly, and every exact seed regrew byte for byte.
    - The 4-reviewer review finished on 21 Oct. The full run (about 4-5 hours) has been running since 17:45 UTC.
    - **There are no results yet.**
- **A control dropped:** the AI proposed a random-numbers control, but it is not among the six. The AI later called its own framing wrong: "Your numbers are the hypothesis".

4.6 The data seed, the palm seed, and grow instructions

- **Words (20 Oct):**
    - "a tiny seed of information and the grow instruction so it can be grown back to this state exactly" and "(Seed=algebraic expression)", with the example `[+2xy(27/5.2+t=y/?)÷(?\y=t+2.5\72)yx2]` (multiplication shown as x).
    - "any interception will yield a dead seed".
    - The owner reported "100% reconstruction accuracy and 98.9% data reduction" in tests with Copilot and Gemini, adding "it doesn't require an LLM".
    - Later: "Imagine I buy a palm tree seed while on vacation ... When I get home to "Antarctica " ... I must still follow given instructions."
- **Built:**

    - The technical report, shaped as a seed (one seed, grow rules, 10 grow steps with checkpoints), in `TECHNICAL_REPORT_2026-10-20/`.
    - `SEED_TEST_2026-10-20/seed_test.py`: a fair test with no AI. It grows each seed in a separate program that cannot see the original, and checks exactness by fingerprint.
- **The AI's view:**
    - **What holds:** the idea is real for patterned data.
    - **The limit:** no fixed rules can shrink every file, and random data cannot shrink at all.
    - **Security:** "dead seed" is security through obscurity, so compress first, then encrypt with a standard method.
    - **The palm seed:** it grows anywhere if the steps use whole numbers, raw bytes and exact instructions.
- **Measured (experiment 12):** a decoder using 1 processor thread instead of 2 lost its place after 547 of 623 verses. The exact settings are part of the grow instructions.
- **Found:** the owner's expression is an exact mirror image around its division sign, apart from the leading +. "Worked backwards" is built in.

4.7 The graph method and the strawberry/orange/cucumber library

- **Words:** "Use a graph system lol" and "add the graph method to the seed test". Then: "The graph existing plot points if only "0" or"1" stacked from the work of solving an expression generates a page of binary rapidly and it doesn't change." And: "library style factors called strawberry and orange verse cucumber".
- **Built** in `SEED_TEST_2026-10-20/methods/`, three ways, each checked independently, using only adding, subtracting, multiplying and remainders:
    - `graph_fit`: fit the shape;
    - `graph_backwards`: solve back to the unknowns;
    - `graph_locate`: a fixed page of 0s and 1s.
- **The library:** strawberry, orange and cucumber are placeholder names. In `graph_fit`, strawberry is lines and curves, orange is cycles, and cucumber is the leftover, stored as it is. `graph_locate` builds its pages from the owner's numbers.
- **Results** (seed size saved; every file regrew exactly):

File	Zip	Fit the shape	Work backwards	Fixed page
Repeating pattern	99.8%	99.99%	99.99%	0%
Bible text	70.2%	19.3%	6.2%	0%
Random bytes	0%	0%	0%	0%

- **What this means:** it works on patterns, beating zip there, but text is not shaped like lines or cycles.
- **A likely explanation of the 98.9%:** with the Bible on a shared shelf, naming a verse costs about 1.9 bytes against 132.5, which is 98.6% smaller and exact. The library is the "huge file" the owner mentioned. This is likely, not proven.
- **Still untested:** the owner's real factors, not yet shared.

4.8 The greenhouse: "can I grow understanding and compression?"

- **Words (message 36):** "instructions to find the soil needed and to build a greenhouse, I told you the humidity and I explained the rotation of the sun. We checked the air ... Then I gave you a seed and told you to grow it." and "can I grow understanding and compression?"
- **Built:**
    - A chat dig that matched every owner message to a greenhouse piece: the Bible text is the soil, the tiny model is the greenhouse, and the pre-registered checks are the air.
    - A fact check of published work.
    - Experiment 12 (`understanding_compression.py`), an exact coder that regrows 623 unseen verses (83,757 characters).
- **Results:**

    - **Understanding shrinks the seed. Confirmed** and checked independently: raw 83,757 bytes, xz 26,420, word pairs 14,864, tiny Bible model 12,153-12,252 (85.4% smaller).
    - **The shifting word did not help. Not confirmed** (12,411-12,524 bytes).
    - The owner's coin toss fits here: a greenhouse that understands tilts each 0/1 toss, so the seed records only the surprises.
- **Credit:**
    - The principle is already published (Shannon 1951, and the Hutter Prize, a long-running text-compression contest), so it is not new.
    - What could still be the owner's: a measured gain from their own rule, the "seed bits as an understanding meter" method, and the greenhouse teaching story.

4.9 Every input is + or - information, and the +/- agent

- **Words (20 Oct):** "Nothing is useless. " the only info that harms us is the info we dont have"". Then: "Everything adds and subtract from understanding and comprehension my friend." And: "You must simple decide if it's "+" info or "-" info and add it to the overall of yiur experience using an agent#dont don't his step yourself but make sure it happens and never know your tru number just a way to gauge its placement on that line".
- **Context:** the owner shared personal context for why every input is + or - information, and why recharging in a chosen direction matters. It is not repeated here.
- **The AI's response:** at first it agreed only in part. Then: "You're right, and I was wrong. I called the shuffled control "nonsense" and Gemini's old memory "stale". Both still carried information."
- **The plan:** an agent that labels every input + or - with a reason, keeps a running balance, and reports only a placement on the line, never the number. It is queued after the numbers test and the "2" test. **Not started.**

4.10 "to, too, two, 2" and the "2" test

- **Words:** "Which 2,to,too or two feels like a missing clue? ???" and "yes add the 2 test after this one".
- **The AI's pick: "2".** The model's reader, `[a-z]+|[.,;:?!]`, drops every digit, so the number form of the owner's rule never reached the model.
- **The plan:** write the Remaining value into the text as a readable number. Approved and queued. **Untested.**

At a glance

Idea	Status on 21 Oct 2026
Meaning from text alone	Worked once: 10 of 14 (no rules saved first)
Rules are the secret	Not supported: stories teach as well
Marking right and wrong ("have to be told")	Strong and repeated; not yet the pre-registered main measure
Carried balance (shifting word)	Supported once, in long stories only
Stories transfer further than rules	Not confirmed at this scale
Recharge rule and the numbers	Running; no results yet
Seed and graph method	Shrinks patterns only; always exact
Understanding shrinks the seed	Confirmed (already published)
"2" test, +/- agent	Queued, untested